

INTE2402/2691 Cloud Security

Assignment 1

Q1.

a.

- Student email:
 - s4129857@student.rmit.edu.au
- 16 bytes = 16 characters
 - s4129857@student
- Convert to ASCII

Character	ASCII (dec)	Hex
s	115	73
4	52	34
1	49	31
2	50	32
9	57	39
8	56	38
5	53	35
7	55	37
@	64	40
s	115	73
t	116	74
u	117	75
d	100	64
e	101	65
n	110	6E
t	116	74

- Round 0 key in hex:
 - 73 34 31 32 39 38 35 37 40 73 74 75 64 65 6E 74

73	39	40	64
34	38	73	65
31	35	74	6E
32	37	75	74

b.

- P = anthony mackenzi fernando 4129857
- First 16 characters:
 - anthony mackenzi
- Convert to ASCII

Char	ASCII	Hex
a	97	61
n	110	6E
t	116	74
h	104	68
o	111	6F
n	110	6E
y	121	79
(space)	32	20
m	109	6D
a	97	61
c	99	63
k	107	6B
e	101	65
n	110	6E
z	122	7A
i	105	69

- Plaintext Matrix

61	6F	6D	65
6E	6E	61	6E
74	79	63	7A
68	20	6B	69

- XOR

61 XOR 73 = 12

6F XOR 39 = 56

6D XOR 40 = 2D

65 XOR 64 = 01

6E XOR 34 = 5A

6E XOR 38 = 56

61 XOR 73 = 12

6E XOR 65 = 0B

74 XOR 31 = 45

79 XOR 35 = 4C

63 XOR 74 = 17

7A XOR 6E = 14

68 XOR 32 = 5A

20 XOR 37 = 17

6B XOR 75 = 1E

69 XOR 74 = 1D

- Initial Transformation Output

12	56	2D	01
5A	56	12	0B
45	4C	17	14
5A	17	1E	1D

c.

- The SubByte Transformation substitutes each byte using the AES S-box
- S-box table: <https://www.redalyc.org/journal/5122/512253718012/html/#gf3>
- Initial Transformation Output

12	56	2D	01
5A	56	12	0B
45	4C	17	14
5A	17	1E	1D

- Substitutions:
 - 12 = C9
 - 56 = B1
 - 2D = D8
 - 01 = 7C
 - 5A = BE
 - 56 = B1
 - 12 = C9
 - 0B = 2B
 - 45 = 6E
 - 4C = 29
 - 17 = F0
 - 14 = FA
 - 5A = BE
 - 17 = F0
 - 1E = 72
 - 1D = A4
- SubByte Transformation output matrix:

C9	B1	D8	7C
BE	B1	C9	2B
6E	29	F0	FA
BE	F0	72	A4

d.

- The ShiftRows transformation performs cyclic left shifts on each row:
 - Row0: no shift
 - Row1: 1-byte shift
 - Row2: 2-byte shift
 - Row3: 3-byte shift
- Final ShiftRow output:

C9	B1	D8	7C
B1	C9	2B	BE
F0	FA	6E	29
A4	BE	F0	72

e.

- Each column is transformed using matrix multiplication in $GF(2^8)$
- MixColumns matrix:

02	03	01	01
01	02	03	01
01	01	02	03
03	01	01	02

- Rules:
 - Multiply by 01: Same value
 - Multiply by 02: Left shift and conditional XOR with 1B
 - Multiply by 03: (02 x value) XOR value
- Column 01: [C9, B1, F0, A4]
 - Element 01:
 - $(02 \times C9) \text{ XOR } (03 \times B1) \text{ XOR } (01 \times F0) \text{ XOR } (01 \times A4)$
 - $(92 \text{ XOR } 1B) \text{ XOR } ((62 \text{ XOR } 1B) \text{ XOR } B1) \text{ XOR } F0 \text{ XOR } A4$
 - 15
 - Element 02:
 - $C9 \text{ XOR } (02 \times B1) \text{ XOR } (03 \times F0) \text{ XOR } A4$
 - $C9 \text{ XOR } 79 \text{ XOR } 0B \text{ XOR } A4$
 - 1F
 - Element 03:
 - $C9 \text{ XOR } B1 \text{ XOR } (02 \times F0) \text{ XOR } (03 \times A4)$
 - $C9 \text{ XOR } B1 \text{ XOR } FB \text{ XOR } F7$
 - 74

- Element 04:
 - $(03 \times C9) \text{ XOR } B1 \text{ XOR } F0 \text{ XOR } (02 \times A4)$
 - $40 \text{ XOR } B1 \text{ XOR } F0 \text{ XOR } 53$
 - 52
- Column 02: [B1, C9, FA, BE]
 - Element 01:
 - $(02 \times B1) \text{ XOR } (03 \times C9) \text{ XOR } FA \text{ XOR } BE$
 - $79 \text{ XOR } 40 \text{ XOR } FA \text{ XOR } BE$
 - $7D$
 - Element 02:
 - $B1 \text{ XOR } (02 \times C9) \text{ XOR } (03 \times FA) \text{ XOR } BE$
 - $B1 \text{ XOR } 89 \text{ XOR } 15 \text{ XOR } BE$
 - 93
 - Element 03
 - $B1 \text{ XOR } C9 \text{ XOR } (02 \times FA) \text{ XOR } (03 \times BE)$
 - $B1 \text{ XOR } C9 \text{ XOR } EF \text{ XOR } D9$
 - $4E$
 - Element 04
 - $(03 \times B1) \text{ XOR } C9 \text{ XOR } FA \text{ XOR } (02 \times BE)$
 - $C8 \text{ XOR } C9 \text{ XOR } FA \text{ XOR } 67$
 - $9C$
- Column 03: [D8, 2B, 6E, F0]
 - Element 01:
 - $(02 \times D8) \text{ XOR } (03 \times 2B) \text{ XOR } 6E \text{ XOR } F0$
 - $AB \text{ XOR } 7D \text{ XOR } 6E \text{ XOR } F0$
 - 48
 - Element 02:
 - $D8 \text{ XOR } (02 \times 2B) \text{ XOR } (03 \times 6E) \text{ XOR } F0$
 - $D8 \text{ XOR } 56 \text{ XOR } B2 \text{ XOR } F0$
 - CC
 - Element 03:
 - $D8 \text{ XOR } 2B \text{ XOR } (02 \times 6E) \text{ XOR } (03 \times F0)$
 - $D8 \text{ XOR } 2B \text{ XOR } DC \text{ XOR } 0B$
 - 24
 - Element 04:
 - $(03 \times D8) \text{ XOR } 2B \text{ XOR } 6E \text{ XOR } (02 \times F0)$
 - $73 \text{ XOR } 2B \text{ XOR } 6E \text{ XOR } FB$
 - CD

- Column04: [7C, BE, 29, 72]
 - Element 01:
 - $(02 \times 7C) \text{ XOR } (03 \times BE) \text{ XOR } 29 \text{ XOR } 72$
 - $F8 \text{ XOR } D9 \text{ XOR } 29 \text{ XOR } 72$
 - 7A
 - Element 02:
 - $7C \text{ XOR } (02 \times BE) \text{ XOR } (03 \times 29) \text{ XOR } 72$
 - $7C \text{ XOR } 67 \text{ XOR } 7B \text{ XOR } 72$
 - 12
 - Element 03:
 - $7C \text{ XOR } BE \text{ XOR } (02 \times 29) \text{ XOR } (03 \times 72)$
 - $7C \text{ XOR } BE \text{ XOR } 52 \text{ XOR } 96$
 - 06
 - Element 04:
 - $(03 \times 7C) \text{ XOR } BE \text{ XOR } 29 \text{ XOR } (02 \times 72)$
 - $84 \text{ XOR } BE \text{ XOR } 29 \text{ XOR } E4$
 - F7
- Final MixColumns

15	7D	48	7A
1F	93	CC	12
74	4E	24	06
52	9C	CD	F7

Q2:

1.

- Encryption exponent (e) is the largest prime factor in the student ID.
- Student ID: $4129857 = 3 \times 3 \times 23 \times 71 \times 281$
- $e = 281$
- Modulus n is 2048 bit
- We can generate p and q of 1024 bit from
https://www.mobilefish.com/services/rsa_key_generation/rsa_key_generation.php
- p (decimal) =
1737940511445775020617313053082388169025126336417660535356158797346
2563170156757520051202390849195020779009155746334930738551176347812
0725921942054837923116860948324791795871798039406573117341872225109
9223830936821855014062228814340432416858093275269349662280420010921
86058197290446870515635897506531387180403
- q (decimal) =
1607529289982713907475685287258115693498109006734569076045322529836
2487261874778266571036357641077382395750100090559216073987518971103
3852277849310110456326797910856531243830955037991402705454696583862
2477216092209903252044345239788187380732419365079571054806591810585
78264987450445592684721472421362469266187
- $n = p \times q$
- n =
2793790276396621391910078113890395970001727183303008002130246574605
7606898932736072696585268877221933141949822101856865048250990938186
8368727052423311448145430971748652748937624472374770396388911381924
4948411014025043786823195625254907076976888009940958196272824162323
9287192383183223261397495810193248308174968618714138840723573109458
3966947281940680309861580664052574582423664330336016004886998942072
3501655027424075011737149754451722115577003646508722687885208495102
9284929839124410295253804622459756950519030977873678384758784363213
0108500777304415825623316969477753691042496074759671782720532911166
26854396933361
- Public Key (n, e) =
(279379027639662139191007811389039597000172718330300800213024657460
5760689893273607269658526887722193314194982210185686504825099093818
6836872705242331144814543097174865274893762447237477039638891138192

4494841101402504378682319562525490707697688800994095819627282416232
 3928719238318322326139749581019324830817496861871413884072357310945
 8396694728194068030986158066405257458242366433033601600488699894207
 2350165502742407501173714975445172211557700364650872268788520849510
 2928492983912441029525380462245975695051903097787367838475878436321
 3010850077730441582562331696947775369104249607475967178272053291116
 626854396933361, 281)

2.

- $e = 281$
- $\varphi(n) = (p - 1)(q - 1)$
- $\varphi(n) =$
 2793790276396621391910078113890395970001727183303008002130246574605
 7606898932736072696585268877221933141949822101856865048250990938186
 8368727052423311448145430971748652748937624472374770396388911381924
 4948411014025043786823195625254907076976888009940958196272824162323
 9287192383183223261397495810193248308174634071733995991830763809624
 3626443419417444966709351052651093255241159287132862426224775067223
 3229251852664819174843002941913026796661545826529586193047264129217
 0103606799421657217855828799663188141546860873170775208932173705807
 5979880979713903185274396252390741869534852842912262858088529337466
 98960540486772
- $d = e^{-1} \bmod \varphi(n)$
- $d =$ 180950117545973342821222141184360166028581618277988418643311343
 9780243578507387176238711216703079854744425219794497490903481025035
 8540940607944320655759274899799593684982315532972943846338840847641
 1068534025197349664757723068186404410351604908760786211527303079714
 4280978743421259553962126101152119278004301702277955725007300070823
 2568092531210795008891082242666043593940375050174469103545043739879
 7988059688198286475832676664247483334831669893715454941169798790772
 7820270055671706313741512370406241530335146451487159103590044717300
 5184992291223597128249010654846240254455606883707263356526751641401
 928466942414123105
- Private Key (n, d) =
 (279379027639662139191007811389039597000172718330300800213024657460
 5760689893273607269658526887722193314194982210185686504825099093818
 6836872705242331144814543097174865274893762447237477039638891138192
 4494841101402504378682319562525490707697688800994095819627282416232

3928719238318322326139749581019324830817496861871413884072357310945
8396694728194068030986158066405257458242366433033601600488699894207
2350165502742407501173714975445172211557700364650872268788520849510
2928492983912441029525380462245975695051903097787367838475878436321
3010850077730441582562331696947775369104249607475967178272053291116
626854396933361,
1809501175459733428212221411843601660285816182779884186433113439780
2435785073871762387112167030798547444252197944974909034810250358540
9406079443206557592748997995936849823155329729438463388408476411068
5340251973496647577230681864044103516049087607862115273030797144280
9787434212595539621261011521192780043017022779557250073000708232568
0925312107950088910822426660435939403750501744691035450437398797988
0596881982864758326766642474833348316698937154549411697987907727820
2700556717063137415123704062415303351464514871591035900447173005184
9922912235971282490106548462402544556068837072633565267516414019284
66942414123105)

3.

- Login information is in the following format:
 - [email], [password], [date and time]
 - s4129857@student.rmit.edu.au, Anthony123, 02/04/2026 15:50:00
- Then the plaintext is converted into a numerical form using ASCII encoding and represented as a large integer M
(https://www.mobilefish.com/services/rsa_key_generation/rsa_key_generation.php):
 - 3188257826149951752085980692108210892860876008179076040214778
4516492321263127421818962507731955516069300981584259629032986
859865142125232415646347210751
- Encryption is performed using RSA:
 - $c = m^e \bmod n$
 - $c =$
783307283201195530435877689810720101486756455735361475641713
1775157606989015039017788069707608891757806560421442931708225
9545123263912248681827723366074788039201648426724628019973822
3526449872729580339853132701400296312272055091133916323811803
7947701465808631269157534026090022934764910465193307341383556
8780202736784873528577944545910786904621001160993918779692943
2100561287557268206715309124731320280717297717609709891913999

2988486798316426935138618243863539696018085216971583385071473
8171687656941134046176563263032210544881437693289160324766791
5051710787510671024479350837079323088477401301583480491757160
7269737

- Then the encrypted ciphertext c is sent to the server

4.

- The server receives the encrypted ciphertext c
- The ciphertext is decrypted using the private key:
 - $m = c^d \bmod n$
- The resulting message m is decided to obtain the original plaintext.
- The server then verifies the login information by:
 - Checking the email address
 - Comparing the received password with the stored password
 - Validating the timestamp
- If all checks are successful, the login is accepted. Otherwise it is rejected.

5.

- **Impersonation Attack**
 - Impersonation attacks occur when an attacker attempts to act as a legitimate user. In this process, confidentiality is preserved by encrypting the login information using the server's public key. But this doesn't fully prevent impersonation attacks because any attacker can encrypt data using the public key which is known by anyone.
 - Therefore, the system relies on password verification at the server to authenticate the users. Encryption protects data confidentiality, but it does not fully prevent impersonation attacks if the attacker has obtained valid credentials.
- **Replay Attack**
 - A replay attack is when an attacker captures a valid encrypted login message and retransmits it to gain unauthorized access. Since the ciphertext can be resent without any modifications the system is vulnerable to replay attacks.
 - However having a timestamp in the login information helps mitigate this risk. If the server validates the latest timestamp and rejects outdated requests, replay attack can be effectively prevented.

Q3:

- $x =$
5506626302227734366957871889516853432625060345377759417550018736038
9116729240
- $y =$
3267051002075881697808308513050704318447127338065924327593890433575
7337482424
- $p =$
1157920892373161954235709850086879078532699846656405640394575840079
08834671663

1.

- The elliptic curve is defined as:

$$y^2 = x^3 + 7 \mod p$$

- Substituting the given values of x , y and p :

$$LHS = y^2 \mod p$$

LHS

$$= 32748224938747404814623910738487752935528512903530129802856995983256684603122$$

$$RHS = (x^3 + 7) \mod p$$

RHS

$$= 32748224938747404814623910738487752935528512903530129802856995983256684603122$$

$$LHS = RHS$$

Therefore, the point (x, y) lies on the given elliptic curve.

(Calculations are done at: <https://www.boxentriq.com/math/modular-exponentiation>,
<https://www.boxentriq.com/math/big-number-calculator>)

2.

- Student ID: s4129857
- Private Key (d) = 4129857
- Public Key is computed using scalar multiplication:

$$Q = d \times p$$

- Using the ECC calculator (<http://www.christelbach.com/ECCalculator.aspx>) with:

- $A = 0$
- $B = 7$
- Modulus p
- The resulting key is:

$$Q = (Qx, Qy)$$

Qx

= 20658619007540524665291049149455512896013416409877263314585095603946553793386

Qy

= 88697329132880878143210789296115344594962296037328102160576650797163995905724

3.

- Instance identity document:

{

“devpayProductCodes”: null,

“marketplaceProductCodes”: [“1abc2defghijklm3nopqrs4tu”],

“availabilityZone”: “us-west-2b”,

“privateIp”: “10.158.112.84”,

“version”: “2017-09-30”,

“instanceId”: “i-1234567890abcdef0”,

“billingProducts”: null,

“instanceType”: “t2.micro”,

“accountId”: “123456789012”,

“imageId”: “ami-5fb8c835”,

“pendingTime”: “2020-07-20M16:32:11Z”,

“architecture”: “x86_64”,

“kernelId”: null,

“ramdiskId”: null,

“region”: “us-west-2”

}

- The instance identity document is hashed using SHA-1
(<https://emn178.github.io/online-tools/sha1.html>):
 - 7e326df5dd59e78cc00fe9511b6205c5f93acdc0
 - e (decimal) = 720457453286302966584138505732100730227221253568
- A random integer k is selected:
 - $k = 123456$
- The value of r is computed as:
 - $r = (k \times p).x \bmod q$
 - $r =$
614527780320051053439634338064931756788928386211860081250479
03224711709993012
- The value of s is computed as:
 - $s = k^{-1} (e + d \cdot r) \bmod q$
 - $A = (e + d \cdot r) \bmod q$
 - $A =$
528000899055864441106144854790127469199675947193890049719141
92858169289379981
 - $B = k^{-1} \bmod q$
 - $B =$
106967181840074197604587870393385739266561088106982695212245
423801198173956265
 - $s = (A \times B) \bmod q$
 - $s =$
355842476158667438165740848134565314668847550445384767559019
34281192616788354
- Therefore, the signature is:
 - (r, s)
 - (6145277803200510534396343380649317567889283862118600812504790
3224711709993012,
3558424761586674381657408481345653146688475504453847675590193
4281192616788354)

4.

- To verify the signature (r, s) ,
- $w = s^{-1} \bmod q$
- $w =$
594956456134241852464752874449706411180904149764062536637031249118
22212098064
- Then:
 - $u_1 = e \cdot w \bmod q$
 - $u_1 = 531939291702013362888214464741638406920740290651174118113$
63306328543116011974
 - $u_2 = r \cdot w \bmod q$
 - $u_2 = 638301331889975724087007919099153311255651190403459212576$
65226160517272138201
- Compute point $X = u_1P + u_2Q$
 - $R1 = u_1P = (x1, y1)$
 - $x1 =$
9285063610379415477099288538167298356171055870374119629601365
0835105717447116
 - $y1 =$
4509344876074188565263380591194125034325138311248511687959846
0652636838872085
 - $R2 = u_2Q = (x2, y2)$
 - $x2 =$
4313188179413396262335467154774425890061147942985953457981716
0790429344016050
 - $y2 =$
3660070997676067783798305863215444967160654902230660014774092
9081114416905257
 - $X = R1 + R2$
 - $X = (Xx, Xy)$
 - $Xx =$
614527780320051053439634338064931756788928386211860081250479
03224711709993012
 - $Xy =$
307388262551704385590735138350047070737884290103392618913123
68323867225258986
- For the verification $Xx \bmod q == r$

- $Xx \bmod q =$
6145277803200510534396343380649317567889283862118600812504790
3224711709993012
- Since $r =$
6145277803200510534396343380649317567889283862118600812504790
3224711709993012
- The signature is valid.

5.

- The ECC NIST P-256 signature provides strong authentication for the instance identity document. The security of the system relies on the difficulty of the Elliptic Curve Discrete Logarithm Problem (ECDLP), which makes it computationally impossible to derive the private key from the public key.
- To forge a valid signature and attacker would need to either:
 - Obtain AWS's private key
 - Successfully solve the ECDLP to reconstruct it from the public key
- Both are computationally infeasible with current technology for the given curve parameters.
- The use of a nonce k in each signature ensures that even if multiple signatures are observed, the private key cannot be derived. Therefore the instance identity document is protected against forgery attacks and any modification to the document or signature will result in verification failure.